

```
+-----+
|       CSC4100       |
|   PROJECT 2   |
| DESIGN DOCUMENT |
+-----+
```

---- GROUP: No Name Group ----

Siyapa Chanhorm <schanhorm42@tntech.edu>

Axel Lopez <alopezmen42@tntech.edu>

Bronson Nguyen <Bhnguyen42@tntech.edu>

Ernesto Paz <eppazsuare42@tntech.edu>

---- PRELIMINARIES ----

>> If you have any preliminary comments on your submission, notes for
>> the TAs, or extra credit, please give them here.

>> Please cite any offline or online sources you consulted while
>> preparing your submission, other than the Pintos documentation,
>> course text, and lecture notes.

ALARM CLOCK

=====

---- DATA STRUCTURES ----

A1:

```
struct sleepingThread {
    struct thread *t;           // Thread that is sleeping
    int64_t wakeup_tick;        // Tick at which it should wake
    struct semaphore sema;      // Used to block/unblock thread
    struct list_elem elem;      // Enables insertion into list
};
```

The purpose for this was to represent a sleeping thread and the tick that should wake up. It enables ordered queuing of sleeping threads and safe blocking/unblocking using semaphores

```
static struct list sleeping_threads;
```

This represents a global list that stores all the sleepingThread structs in an ascending wake-up order to allow efficient wakeup checking inside the timer interrupt.

---- ALGORITHMS ----

>> A2: Briefly describe what happens in a call to timer_sleep(),

>> including the effects of the timer interrupt handler.

- When a thread calls timer_sleep() is used to calculate wake-up time by adding it timer_ticks() + ticks(the amount the thread passes), and then adds itself to a ordered sleeping thread list. The thread then blocks itself to release its CPU usage for other thread to run
- The timer interrupt handler increments the global tick count and then checks the sleeping list to wake up any thread it time has arrived to be unblocked.

>> A3: What steps are taken to minimize the amount of time spent in

>> the timer interrupts the handler?

- Uses an ordered list to insert threads into the sleep list minimizing time spent checking the entire list. Since the sleeping threads list is ordered the interrupt handler only needs to check the earliest wake-up thread instead of checking every thread on the list.

---- SYNCHRONIZATION ----

>> A4: How are race conditions avoided when multiple threads call

>> timer_sleep() simultaneously?

- Race conditions are avoided by temporarily disabling interrupts when modifying shared data structure, so when a thread uses timer_sleep() it disables interrupts then inserts itself into the sleep_list this ensures that no other threads or interrupt handler can modify the shared list at the same time to avoid any errors

>> A5: How are race conditions avoided when a timer interrupt occurs

>> during a call to timer_sleep()?

- To avoid race conditions when a timer interrupt occurs during a call to timer_sleep() implement semaphores and locks to protect access to shared data for the sleeping thread list.
 - When either timer_sleep or interrupt handler is called it uses a lock before modifying sleeping thread list to ensure the list isn't access at the same time
 - Usage of locks around shared data to make sure that only one thread can access the data time

---- RATIONALE ----

>> A6: Why did you choose this design? In what ways is it superior to

>> another design you considered?

- This design is better because it ensures mutual exclusion to prevent race conditions making it more effective, making it more effective. It ensures the list won't be accessed concurrently by multiple threads while other designs like busy waiting or disabling interrupts would introduce performance issues wasting CPU usage.

PRIORITY SCHEDULING

=====

---- DATA STRUCTURES ----

>> B1: Copy here the declaration of each new or changed `struct' or

>> `struct' member, global or static variable, `typedef', or

>> enumeration. Identify the purpose of each in 25 words or less.

In threads.h wa added this to struct.thread:

```
int original_priority;
    // The thread's base priority before any donations; used to restore on
    lock release.
```

```
struct list_donations;
    // List of threads that have donated to this thread (supports nested
    donations).
```

```
struct list_elem donation_elem;
    // Node linking this thread into another's donations list when it
    donates.
```

```
bool donated;
    // True if this thread's priority was elevated by a donation.
```

```
int priorities[MAX_DONORS];
    // Stack of donated priority values, for quick removal when locks are
    released.
```

```
int donation_no;
    // Count of active donations (auxiliary for managing priorities[]).
```

```
struct lock *waiting_for;
    // The lock this thread is blocked on; used to propagate donations up
    the chain.
```

Plus in src/threads/thread.c:

```
extern struct list ready_list;
// Global ready-queue, always sorted by effective priority.
```

- So what we did here is we extended struct thread with fields to manage priority donation: original_priority stores the thread's base priority; donations and its donation_elem link in donor threads; the priorities[] stack and donation_no counter record and pop donated values; donated flags when a donation is active; waiting_for points to the lock causing the block; and we use a global ready_list sorted by each thread's effective (possibly donated) priority.

>> B2: Explain the data structure used to track priority donation.
 >> Use ASCII art to diagram a nested donation. (Alternately, submit a
 >> .png file.)

- We keep, in each thread's struct (thread.h), a list of its donors:

```
struct thread {
    ...
    struct list donations;           // threads that have donated to me
    struct list_elem donation_elem; // node when I appear in another's
donations
    struct lock *waiting_for;       // lock I'm blocked on (for propagating
donation)
    int priorities[MAX_DONORS];     // stack of donated priorities
    int donation_no;               // count of active donations
    bool donated;                  // true if my priority was elevated
    ...
};
```

BEFORE ANY DONATION

```
+-----+ +-----+ +-----+
| T1:60 | | T2:30 | | T3:20 |
+-----+ +-----+ +-----+
```

AFTER T₁ BLOCKS ON L₁

```
+-----+ +-----+ +-----+
| T1:60 |---L1---> | T2:60* | | T3:20 |
```

```

+-----+      +-----+      +-----+
      (donations = [T1])

```

```

      AFTER T2 BLOCKS ON L2
+-----+      +-----+      +-----+
| T1:60 |---L1-->| T2:60 |---L2-->| T3:60* |
+-----+      +-----+      +-----+
      (donations=[T1]) (donations=[T2])

```

Diagram explanation:

In the first frame, all threads run at their base priorities. After **T₁** (prio 60) blocks on **L₁** held by **T₂**, it donates its priority (marked by *) and appears in **T₂**'s donations list. Next, **T₂** (now prio 60) blocks on **L₂** held by **T₃**, cascading the same donation so **T₃** also jumps to 60.

- **Boxes** show each thread's *effective* priority (* = donation).
- **Arrows** represent “waiting on” relationships (locks **L₁**, **L₂**).
- **donations=[...]** lists who has donated to that thread.

---- ALGORITHMS ----

>> B3: How do you ensure that the highest priority thread waiting for

All wait-lists are kept in descending priority order:

1. Semaphores (`synch.c`):

```

In sema_down():
    list_insert_ordered(&sema->waiters, &cur->elem, sema_priority_cmp,
NULL);
In sema_up():
    list_sort(&sema->waiters, sema_priority_cmp, NULL);
    t = list_entry(list_pop_front(&sema->waiters), struct thread, elem);
    thread_unblock(t);

```

2. Condition variables (`synch.c`):

```

In cond_wait():
    list_insert_ordered(&cond->waiters, &waiter.elem,

```

```
cond_sema_priority_cmp, NULL);
In cond_signal():
    se = list_entry(list_pop_front(&cond->waiters), struct
semaphore_elem, elem);
    sema_up(&se->semaphore);
```

3. Ready queue (`thread.c`):

```
thread_unblock() and thread_yield() use
list_insert_ordered(&ready_list, &t->elem, compare_priority, NULL);
```

This ensures that the highest-priority thread is always the one to run next. By keeping all wait-lists—semaphores, condition variables, and the ready queue—sorted in descending priority order, we guarantee that the thread at the front is always the most important one. This prevents low-priority threads from running ahead of high-priority ones and helps maintain fairness and responsiveness in scheduling.

>> B4: Describe the sequence of events when a call to lock_acquire()
>> causes a priority donation. How is nested donation handled?

In `lock\_acquire(lock)` (`synch.c`):

```
struct thread *cur = thread_current();
if (lock->holder != NULL && cur->priority > lock->holder->priority) {
    /* donate cur->priority up the chain */
    cur->waiting_for = lock;
    donate_to(lock->holder, cur->priority);
}
sema_down(&lock->semaphore);
lock->holder = cur;
```

>> B5 Describe the sequence of events when lock_release() is called

>> on a lock that a higher-priority thread is waiting for.

1. Assert lock held by current thread; set lock->holder = NULL.
2. Walk current->donations, remove any donors whose waiting_for == lock, clear their donated flag.
3. Recompute current->priority = max(original_priority, any remaining donations).
4. Call sema_up(&lock->semaphore), which list_sorts waiters and unblocks the highest-priority thread.
5. Invoke thread_maybe_yield() to preempt if the newly unblocked thread has higher priority.

---- SYNCHRONIZATION ----

>> B6: Describe a potential race in `thread_set_priority()` and explain
>> how your implementation avoids it. Can you use a lock to avoid
>> this race?

- This section addresses a potential race condition that could occur if a thread's priority is changed while it's being rescheduled. For example, if an interrupt or another thread preempts execution during `thread_set_priority()`, the ready list could be left in an inconsistent state. To avoid this, we disable interrupts at the start of `thread_set_priority()` using `intr_disable()`, perform all updates to the thread's `original_priority`, `priority`, and ready list atomically, and re-enable interrupts only after the changes are complete. This ensures the scheduler maintains a consistent view of thread priorities without needing a global lock.

---- RATIONALE ----

>> B7: Why did you choose this design? In what ways is it superior to
>> another design you considered?

- The design prioritizes correctness, performance, and modularity. Using deferred yielding (`intr_yield_on_return()`) avoids calling `thread_yield()` directly in interrupt context, which simplifies preemption handling. Priority changes trigger list operations like insertion or sorting, but these are only $O(n)$ and occur infrequently. Each thread manages its own donation state, and the MLFQS scheduler can be toggled on or off cleanly with `thread_mlfqs`. By always choosing the highest-priority thread to run and keeping interrupt-disabled sections minimal, our solution remains fair, efficient, and easy to reason about.

ADVANCED SCHEDULER =====

---- DATA STRUCTURES ----

>> C1: Copy here the declaration of each new or changed `'struct'` or
>> `'struct'` member, global or static variable, `'typedef'`, or
>> enumeration. Identify the purpose of each in 25 words or less.

In `thread.h`

```
typedef int fixed_t;
```

```

    // 17.14 fixed-point type for load_avg and recent_cpu.

#define F (1 << 14)
    // Scaling factor for 14 fractional bits.

extern fixed_t load_avg;
    // System load average in 17.14 format, updated once per second.

extern bool thread_mlfqs;
    // True if MLFQS is in use (disables priority donation).

struct thread {
    int nice;
    // Niceness (-20..20), shifts priority by nice*2.

    fixed_t recent_cpu;
    // Fixed-point "recent CPU" usage, increments each tick.
};

```

In src/threads/thread.c:

```

static struct list all_list;
    // All threads, used when recalculating recent_cpu & priorities.

extern struct list ready_list;
    // Ready queue, always sorted by descending priority after MLFQS
    recalc.

```

---- ALGORITHMS ----

>> C2: Suppose threads A, B, and C have nice values 0, 1, and 2. Each
>> has a recent_cpu value of 0. Fill in the table below showing the
>> scheduling decision and the priority and recent_cpu values for each
>> thread after each given number of timer ticks:

Threads A, B, C start with recent_cpu = 0 and nice = 0,1,2. Priorities computed as
PRI_MAX – recent_cpu/4 – nice*2, clamped to [PRI_MIN..PRI_MAX]. They run as:

timer	recent_cpu	priority	thread to run				
ticks	A	B	C	A	B	C	
0	0	0	0	63	61	59	A
4	4	0	0	62	61	59	A

8	8	0	0	61	61	59	B
12	8	4	0	61	60	59	A
16	12	4	0	60	60	59	B
20	12	8	0	60	59	59	A
24	16	8	0	59	59	59	B
28	16	12	0	59	58	59	A
32	20	12	0	58	58	59	C
36	20	12	4	58	58	57	C

(Values adjust if PRI_MAX=63. Behavior matches tests in tests/threads/mlfqs-fair-.c.)*

>> C3: Did any ambiguities in the scheduler specification make values

>> in the table uncertain? If so, what rule did you use to resolve

>> them? Does this match the behavior of your scheduler?

- When two threads share priority, we rely on our stable `list_insert_ordered`: the one inserted first (older in `ready_list`) runs first (FIFO). This matches test assumptions and our implementation in `thread_unblock()` and `thread_yield()`

>> C4: How is the way you divided the cost of scheduling between code

>> inside and outside interrupt context likely to affect performance?

Interrupt handler (`timer_interrupt` in `devices/timer.c`) only:

- increments `recent_cpu`
- sets flags for periodic recalc
- calls `intr_yield_on_return()` (never `thread_yield()`).

Full recalculation (`load_avg`, all `recent_cpu`, all priorities, `sort_ready_list`)

happens in-line but defers the actual context switch until after the ISR returns.

This keeps the interrupt path short and defers scheduling overhead to normal thread context.

---- RATIONALE ----

>> C5: Briefly critique your design, pointing out advantages and

>> disadvantages in your design choices. If you were to have extra

>> time to work on this part of the project, how might you choose to

>> refine or improve your design?

Advantages:

- Automatic proportional-share scheduling via MLFQS formulas.
- No manual priority calls when MLFQS is enabled.
- Deferred yields and minimal interrupt work keep latency low.

Disadvantages:

- $O(n)$ scan of ``all_list`` every second.
- Fixed-point math per tick adds CPU overhead.
- Right now, when a thread wakes up from blocking we don't recalculate its priority right away so it can run with an out-of-date priority which is why the mlfqs-block test is still failing

Future improvements:

- Incremental/in-place updates to reduce $O(n)$ costs.
- Adjustable recalculation intervals for large ``all_list``.
- Hybrid scheduler combining MLFQS and priority donation.

>> C6: The assignment explains arithmetic for fixed-point math in detail, but it leaves it open to you to implement it. Why did you decide to implement it the way you did? If you created an abstraction layer for fixed-point math, that is, an abstract data type and/or a set of functions or macros to manipulate fixed-point numbers, why did you do so? If not, why not?

We implemented fixed-point arithmetic as macros (``INT_TO_FP``, ``ADD_FP``, ``MUL_FP``, etc.) in ``thread.h`` to:

- Inline operations for speed (no function call overhead).
- Keep code simple and transparent for testing.
- Match exact 17.14 specification from the assignment.

An abstraction layer was avoided to minimize complexity and call overhead in the timing path.

SURVEY QUESTIONS

=====

Answering these questions is optional, but it will help us improve the course in future quarters. Feel free to tell us anything you want--these questions are just to spur your thoughts. You may also choose to respond anonymously in the course evaluations at the end of the quarter.

>> In your opinion, was this assignment, or any one of the three problems in it, too easy or too hard? Did it take too long or too little time? It took us very long so yes it was hard and especially 4.5.

>> Did you find that working on a particular part of the assignment gave

>> you greater insight into some aspect of OS design?

Yes it gave me a recap of my Data Structures class and made me more experienced with threads.

>> Is there some particular fact or hint we should give students in

>> future quarters to help them solve the problems? Yes especially some help for 4.5 to help solve problems

Conversely, did you

>> find any of our guidance to be misleading?

>> Do you have any suggestions for the TAs to more effectively assist

>> students, either for future quarters or the remaining projects?

>> Any other comments?